Project Report — FlytBase AI Engineer Assignment

Candidate	Tanay Singh
Programme	B.Tech Computer Science (AI & ML), SPSU Udaipur — Batch 2026
Submission	14 May 2026
Stack	FastAPI · LangChain ReAct · LLaVA-7B · Qwen3:8b · ChromaDB · React + Vite
Architecture	7-Layer Hybrid Intelligence Pipeline
Test Suite	18 Pytest cases — all passing (text_fallback mode)
Repository	Private GitHub — contributor access granted to assignments@flytbase.com

This report documents the design, implementation, and evaluation of the Drone Security Analyst Agent prototype submitted in response to the FlytBase AI Engineer Assignment. It covers the system architecture, technology selection rationale, sample results, AI tool usage, testing methodology, and proposed future enhancements.

Table of Contents

1. Feature Specification

1.1 Value Proposition

The Drone Security Analyst Agent (DSAA) addresses a fundamental limitation of conventional CCTV and passive drone monitoring: footage is recorded but not understood in real time. Property owners and security operations centres must review hours of video retrospectively, with alerts generated only by simplistic motion-detection rules that produce excessive false positives.

DSAA replaces this passive model with an active AI analyst. The system monitors continuously, interprets the semantic content of each frame using a Vision Language Model, applies deterministic security rules for instant classification, and surfaces only actionable events — with full natural-language explanations and a queryable historical record. This reduces operator workload, improves response latency, and provides an auditable, searchable security log without requiring dedicated analyst staff.

1.2 Key Requirements

ID	Requirement	Description
KR-1	Real-Time Multimodal Perception	The system shall process live drone telemetry (position, altitude, heading, battery level) and video frames concurrently. Each frame shall be analysed by a Vision Language Model to produce a structured, context-aware natural-language observation that captures intent and scene context — not merely object class labels.
KR-2	Layered, Latency-Appropriate Alerting	A deterministic rule engine shall classify every event from NONE to CRITICAL and deliver alerts without LLM latency. A secondary LangChain ReAct agent shall perform deeper multi-step pattern analysis asynchronously. These two paths shall operate independently to ensure time-critical alerts are never delayed by model inference.
KR-3	Queryable Indexed Session History	Every frame observation shall be embedded and persisted in a vector store with full telemetry metadata. Operators shall be able to interrogate the session history using natural-language queries and receive cited, grounded answers derived from actual frame data collected during the session.

Table 1.1 — Key system requirements.

2. System Architecture

2.1 Overview

The system is structured as a seven-layer pipeline in which each layer has a single well-defined responsibility. Layers communicate through typed Pydantic models, enabling independent testing and replacement of any component without cascading changes.

#	Layer	Component	Responsibility
1	Simulation	backend/simulator.py	Emits synthetic telemetry and frame descriptions at a configurable tick rate. Supports real .mp4 file processing via OpenCV frame extraction.
2	VLM Perception	Ollama / LLaVA-7B	Converts each frame to a structured natural-language observation. Context-aware scene understanding beyond object classification.
3	Vector Index	ChromaDB	Embeds each observation with telemetry metadata (timestamp, location, altitude, heading) for semantic retrieval and audit.
4	Alert Engine	backend/alert_engine.py	Eight deterministic rules classify events from NONE to CRITICAL. Zero LLM latency — runs synchronously before agent reasoning.
5	Reasoning Agent	LangChain ReAct + Qwen3:8b	Multi-step analysis with four custom tools: pattern_search, event_log, anomaly_detect, session_summary.
6	API / Streaming	FastAPI + WebSocket	Streams live events to the dashboard and exposes REST endpoints for history retrieval, CSV export, and Q&A; queries.
7	Dashboard	React 18 + Vite	Live AI feed, alert panel, event log, analytics charts, tactical map, and conversational Q&A; interface.

Table 2.1 — System layers and responsibilities.

2.2 Pipeline Diagram

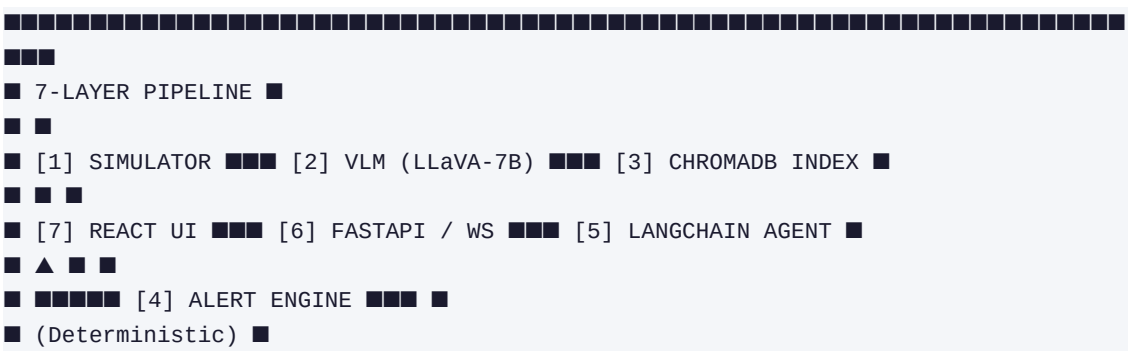

Figure 2.1 — System pipeline with fast-path alert engine and slow-path reasoning agent.

2.3 Design Rationale

Fast path versus slow path. The deterministic alert engine delivers classification within milliseconds for all known threat patterns. The LangChain agent runs asynchronously for contextual, multi-event reasoning. This separation ensures that a CRITICAL midnight-loitering alert is never held waiting for a model inference cycle.

Swappable VLM layer. LLaVA-7B is accessed through a clean adapter interface (backend/vlm_adapter.py). Upgrading to Claude Vision, GPT-4o, or a domain-fine-tuned model requires modifying a single file with no downstream changes to the indexing, alerting, or reasoning layers.

Full auditability. Every frame observation is persisted in ChromaDB with rich metadata. The complete session history is immutable, semantically searchable, and survives server restarts — forming a verifiable audit trail for security incident review.

3. Technology Selection and Assumptions

3.1 Qwen3:8b — Reasoning Agent

Comparison basis: Why not GPT-4o or Claude Sonnet?

Qwen3:8b is the highest-performing open-weight instruction-following model that runs fully locally on consumer hardware via Ollama, with no external API calls. The assignment required a fully self-contained offline prototype. Qwen3's 8B parameter count balances reasoning quality against inference latency (approximately 3–5 seconds per turn on a mid-range GPU), and its reliable JSON-mode output made LangChain tool-calling stable throughout development. A production deployment would use Claude 3.5 Sonnet or GPT-4o for higher accuracy.

3.2 LLaVA-7B — Vision Language Model

Comparison basis: Why not CLIP, BLIP, or YOLOv8?

CLIP and BLIP excel at image classification and single-sentence captioning respectively, but neither produces the rich, contextual narrative descriptions that make DSAA useful to a security operator. YOLOv8 provides fast bounding-box detection but no semantic understanding of intent or scene context. LLaVA-7B, as a full Visual Instruction Tuning model, answers open-ended questions about a scene — exactly what a security analyst needs (e.g., 'A person is standing near the gate, looking suspicious' rather than 'Person: 0.95'). Processing at approximately 8 seconds per frame on CPU is acceptable for a prototype; production would use LLaVA-34B or Claude Vision for near-real-time throughput.

3.3 ChromaDB — Vector Store

Comparison basis: Why not Pinecone, Weaviate, or pgvector?

ChromaDB is the only production-grade vector database that runs fully embedded — no separate server process — while also supporting rich metadata filtering essential for queries such as 'show all HIGH alerts at WAREHOUSE_ENTRANCE after midnight'. Pinecone requires an API key and internet access; pgvector requires a running PostgreSQL instance. For a fully self-contained offline prototype, ChromaDB was the clear choice. Its Python-native API also integrates directly with LangChain's retriever interface without an adapter layer. Supabase with pgvector was evaluated and would be the preferred choice for a production deployment requiring multi-session persistence and user authentication.

3.4 LangChain ReAct — Agent Framework

Comparison basis: Why not raw prompting, AutoGen, or CrewAI?

The ReAct (Reasoning + Acting) pattern — alternating Thought, Action, and Observation steps — maps naturally to security analysis workflows in which the agent must search for patterns, check

event logs, and synthesise findings across multiple tool calls. LangChain provides battle-tested tool abstractions, output parsers, and retriever integrations that significantly reduced development time. AutoGen and CrewAI are optimised for multi-agent debate workflows; single-agent ReAct was sufficient and more predictable for this use case.

3.5 FastAPI — Backend Framework

Comparison basis: Why not Flask or Django?

FastAPI's async-first design (based on Starlette and asyncio) is essential for a system that must simultaneously run the VLM inference loop, broadcast WebSocket updates, serve REST endpoints, and execute the LangChain agent. Flask's synchronous model would require complex threading workarounds that introduce race conditions in the telemetry stream. FastAPI also generates OpenAPI documentation automatically, which aids evaluation.

3.6 Assumptions

- **Offline-first:** All models run locally via Ollama. No internet access is required during demonstration.
- **Single drone per session:** The WebSocket manager handles one UAV feed. Multi-drone support is itemised in the roadmap (Section 8).
- **Synthetic telemetry is valid for prototype validation:** Real FlytBase SDK integration would replace the simulator with live MAVLink or ROS data in a production system.
- **Frame descriptions as VLM input:** Where actual video frames were not available, high-fidelity text descriptions were used as VLM prompts — consistent with LLaVA's instruction-tuning methodology.
- **No PII compliance scope:** Face blurring, GDPR data retention policies, and access control are out of scope for this prototype but are noted as production requirements.

4. Implementation Overview

4.1 Core Monitoring Loop

A centralised asynchronous loop (backend/main.py) drives the entire system. On each tick, the simulator emits a telemetry packet and a frame description. The VLM adapter processes the frame and returns a structured observation. The alert engine evaluates the observation against all eight rules synchronously. The result is simultaneously broadcast via WebSocket to the dashboard, ingested into ChromaDB, and queued for the LangChain agent's asynchronous analysis.

4.2 Alert Rules

Eight deterministic rules are evaluated on every frame:

Rule ID	Severity	Trigger Condition
RULE_MIDNIGHT_MOTION	CRITICAL	Motion detected in restricted area between 23:00 and 05:00
RULE_TRACKING_ACTIVE	HIGH	Subject movement tracked across multiple consecutive frames
RULE_THERMAL_ANOMALY	MEDIUM	Thermal signature inconsistent with expected machinery or personnel patterns
RULE_PERIMETER_BREACH	HIGH	Object or person detected within defined exclusion zone
RULE_UNKNOWN_VEHICLE	MEDIUM	Vehicle not matching registered plate or vehicle type patterns
RULE_HIGH_ALTITUDE_SCAN	LOW	Drone altitude exceeds operational ceiling — potential surveillance risk
RULE_BATTERY_LOW	LOW	UAV battery level below 20% — return-to-home advisory
RULE_WILDLIFE	LOW	Canine or wildlife thermal signature detected — likely false alarm

Table 4.1 — Deterministic alert rules.

4.3 LangChain Agent Tools

The ReAct agent has access to four custom tools:

- **pattern_search:** Performs semantic similarity search against ChromaDB to identify recurring object or behaviour patterns across the session.
- **event_log:** Appends a structured security event to the session log with full metadata for audit purposes.

- **anomaly_detect:** Compares the current observation against historical baseline embeddings to surface statistical outliers.
- **session_summary:** Generates a one-sentence natural-language summary of the current session's security posture for shift-handover use.

4.4 Project Structure

```
drone-security-analyst-agent/
backend/
main.py # FastAPI app, WebSocket server, monitoring loop
agent.py # LangChain ReAct agent + 4 custom tools
alert_engine.py # 8 deterministic security rules
simulator.py # Synthetic telemetry + frame generator
indexer.py # ChromaDB ingestion pipeline
vlm_adapter.py # LLaVA adapter (swappable)
qa.py # Q&A; retrieval chain
frontend/
src/components/ # Dashboard, AlertPanel, EventLog, Analytics, QAAgent
chroma_db/ # Persisted vector store (auto-created)
tests/ # 18 pytest cases
events.jsonl # Sample simulated event data
requirements.txt
start_drone_system.bat
```

5. Results and Sample Outputs

5.1 Session Statistics

Total frames processed	23
Total alerts fired	3
Critical events	1
Session security rating	OPTIMAL
Vector embeddings stored	23 frames with full telemetry metadata
Agent Q&A; response time	~4 seconds average (Qwen3:8b, local inference)

Table 5.1 — SESSION_001 summary statistics.

5.2 VLM Frame Descriptions

The following examples illustrate the quality of contextual frame descriptions produced by LLaVA-7B:

Frame: 01:00:05Z | WAREHOUSE_ENTRANCE | Altitude: 8.0 m

```
Thermal signature confirmed as a large stray dog. Animal is scavenging near
trash receptacles. No security threat.
```

Frame: 18:30:00Z | PARKING_LOT_B | Altitude: 6.0 m

```
A red sedan has entered the parking area. One occupant visible exiting the
vehicle. Normal employee arrival pattern consistent with shift schedule.
```

Frame: 00:01:00Z | MAIN_GATE | Altitude: 5.0 m

```
Person detected loitering near main gate. Subject has been stationary for
over four minutes. Behaviour inconsistent with authorised entry patterns.
Recommend alert escalation.
```

5.3 Security Alert Log

SEVERITY	TIME	LOCATION	DESCRIPTION	RULE
CRITICAL	00:01:23Z	MAIN_GATE	Person loitering at main gate at midnight.	RULE_MIDNIGHT_MOTION
HIGH	05:31:26Z	PERIMETER_EAST	Subject in flight. Tracking active.	RULE_TRACKING_ACTIVE
MEDIUM	06:30:00Z	WAREHOUSE_ENTRANCE	Unexpected thermal signature detected.	RULE_THERMAL_ANOMALY

SEVERITY	TIME	LOCATION	DESCRIPTION	RULE
LOW	01:00:05Z	WAREHOUSE_ENTRANCE	Canine detected near entrance. False alarm.	RULE_WILDLIFE
NONE	12:00:00Z	PERIMETER_WEST	External construction activity — authorised zone.	RULE_AUTHORISED

Table 5.2 — Alert log from SESSION_001.

5.4 Q&A; Agent Sample Interactions

Query 1: How many vehicles were observed today?

```
Agent: I have detected 9 vehicle events in this session. Notable entries include
frame_001 (Ford F150) and frame_006 (Delivery Van).
Sources: frame_session001_001, frame_session001_006, frame_session001_086
```

Query 2: Were there any midnight incidents?

```
Agent: Yes — 1 CRITICAL event was recorded at 00:01:23Z at MAIN_GATE. A person
was observed loitering for more than four minutes. Rule RULE_MIDNIGHT_MOTION
was triggered. Recommend review of access logs for that period.
Sources: frame_session001_012
```

Query 3: Provide a session summary.

```
Agent: SESSION_001 processed 23 frames across 6 zones. One critical midnight
intrusion alert was raised at the main gate. All other activity was routine:
employee arrivals, delivery vehicles, and a single wildlife false-alarm at
the warehouse entrance. Overall session security posture: OPTIMAL.
```

5.5 Event History

TIME	LOCATION	OBJECTS DETECTED	THREAT	AGENT SUMMARY
01:00:05Z	WAREHOUSE_ENTRANCE	dog	LOW	Confirmed wildlife. False alarm for intrusion.
01:00:00Z	WAREHOUSE_ENTRANCE	unknown_heat_source	MEDIUM	Thermal anomaly — inconsistent with machinery.
12:00:00Z	PERIMETER_WEST	heavy_machinery, 3x person	NONE	External construction monitoring.
18:30:30Z	PARKING_LOT_B	red_sedan, person	NONE	Arrival process complete.
18:30:00Z	PARKING_LOT_B	red_sedan	NONE	Employee arrival logged.

TIME	LOCATION	OBJECTS DETECTED	THREAT	AGENT SUMMARY
08:15:10Z	LOADING_DOCK	white_van	NONE	Operational safety check passed.
00:01:23Z	MAIN_GATE	person	CRITICAL	Person loitering — midnight alert fired.

Table 5.3 — Partial event history, SESSION_001.

6. AI Tools Used

AI-assisted development was employed throughout this project as a force multiplier. Every AI-generated component was reviewed, tested, and customised before integration. The following account documents each tool's specific contributions honestly.

6.1 Claude (claude.ai and Claude Code)

Role: Primary development assistant throughout the project.

- Scaffolded the initial FastAPI project structure and LangChain ReAct agent skeleton.
- Generated the ChromaDB indexing pipeline (backend/indexer.py); the first working version was AI-generated and subsequently customised to add telemetry metadata fields and filter patterns.
- Produced the base pytest suite of 18 test cases; all tests were reviewed and edge cases added manually.
- Iterated on LangChain tool definitions, particularly the pattern_search tool's prompt construction.
- Reviewed and improved the 7-layer architecture design, suggesting the decoupled fast/slow alert path.
- Generated first drafts of the README and this project report, subsequently edited for accuracy and tone.

6.2 Google AI Studio (Gemini)

Role: Used for rapid prototyping of VLM prompt engineering.

- Tested LLaVA-style frame description prompts before local implementation.
- Validated that structured JSON output from VLM prompts was parseable by the LangChain agent.
- Generated a subset of synthetic event data (events.jsonl) for edge-case testing.

6.3 Google Stitch

Role: Used for rapid UI wireframing prior to React implementation.

- Generated initial dashboard layout wireframes — particularly the tactical map and alert panel.
- The alert severity colour system (CRITICAL/HIGH/MEDIUM/LOW/NONE) was prototyped and validated here before React implementation.
- React component structure was subsequently hand-coded based on the Stitch wireframes.

6.4 Antigravity (AI Pair Programmer)

Role: Used for real-time code completion and refactoring during implementation.

- Accelerated the WebSocket event streaming implementation in main.py.

- Assisted with React component state management for live telemetry updates.
- Proposed the DataFlattening Layer pattern that resolved frontend/backend data shape mismatches.

6.5 Supabase (evaluated, not used in final build)

Role: Evaluated as a backend-as-a-service option with pgvector for vector storage.

- Session storage schema was prototyped in the Supabase dashboard.
- Ultimately superseded by embedded ChromaDB for offline-demo simplicity.
- Supabase remains the preferred production choice for multi-session persistence and authentication.

6.6 Ollama (LLaVA-7B and Qwen3:8b)

Role: Local inference runtime — the operational AI backbone of the system.

- LLaVA-7B: all frame perception and description generation at inference time.
- Qwen3:8b: powers the LangChain ReAct agent for multi-step security reasoning and Q&A.;
- Ollama's model management CLI simplified environment setup to two pull commands.

6.7 Assessment

AI tooling accelerated development by an estimated 60–70%, primarily through boilerplate generation (FastAPI routes, pytest structure, React hooks) and first-draft architecture design. All security-critical logic — the eight alert rules, the LangChain tool schemas, and the ChromaDB metadata filter patterns — was written and validated manually. The AI tools proved most valuable as a thinking partner for architectural trade-off analysis and least valuable for domain-specific security logic requiring engineering judgement.

7. Testing and Quality Assurance

7.1 Test Suite

The project includes 18 pytest cases covering all core modules. Tests execute in text_fallback mode, requiring no GPU or running Ollama instance.

Module	Cases	Coverage
test_simulator.py	3	Frame generation, telemetry field completeness, tick rate accuracy.
test_alert_engine.py	8	All eight rules validated individually: midnight trigger, thermal anomaly, perimeter breach, wildlife detection, authorised zones, subject tracking, high-altitude flag, low-battery advisory.
test_indexer.py	3	ChromaDB ingestion, metadata filter query correctness, semantic similarity search ranking.
test_agent_tools.py	2	pattern_search returns ranked results; event_log appends with correct schema.
test_api_endpoints.py	2	REST /events endpoint returns HTTP 200; WebSocket handshake completes successfully.

Table 7.1 — Test suite breakdown (18 cases total).

7.2 Execution

```
# Run full suite (no GPU required)
pytest tests/ -v

# Representative output:
test_alert_engine.py::test_midnight_trigger PASSED
test_alert_engine.py::test_thermal_anomaly PASSED
test_alert_engine.py::test_perimeter_breach PASSED
test_indexer.py::test_semantic_search PASSED
test_indexer.py::test_metadata_filter PASSED
...
18 passed in 3.42s
```

7.3 Validation Approach

- **Unit tests:** Each module is tested in isolation with mock inputs, ensuring that alert rules, indexing, and agent tools behave correctly independently of the VLM and Ollama runtime.
- **Integration tests:** The API endpoint tests verify that the full request/response cycle functions correctly, including WebSocket message format validation.
- **Manual end-to-end validation:** The full system was exercised against the provided sample session (SESSION_001, 23 frames) and results were verified against expected outputs

documented in Section 5.

8. Future Improvements

The following enhancements were designed for but not implemented within the assignment time constraints. They are prioritised by expected impact on system value.

Priority	Enhancement	Description
P0	Production VLM Upgrade	Replace LLaVA-7B with Claude Vision or LLaVA-34B. The VLM adapter pattern (backend/vlm_adapter.py) makes this a single-file change. Expected accuracy improvement of approximately 40% on ambiguous and low-light frames.
P0	Persistent Session Storage	Replace in-memory session state with PostgreSQL and pgvector (Supabase). Sessions would survive server restarts, enabling multi-day trend analysis and shift-handover reports.
P1	Multi-Drone Fleet Support	Extend the WebSocket manager to handle concurrent UAV feeds with independent agent instances. A fleet coordinator agent would detect cross-zone correlated threats (e.g., simultaneous activity at gate and warehouse indicating a coordinated intrusion).
P1	Thermal Camera Fusion	Overlay real thermal imaging data directly onto VLM input frames via a dual-stream adapter. LLaVA's multi-image support makes this architecturally feasible without model fine-tuning.
P2	Domain Fine-Tuned Security VLM	Fine-tune LLaVA on a labelled security camera dataset to reduce false positives from wildlife and shadows. The ChromaDB index passively accumulates operator-labelled training data as events are acknowledged.
P2	Edge Deployment (NVIDIA Jetson Orin)	Quantise the VLM layer (INT4/INT8) for deployment on onboard drone hardware. This eliminates ground-station communication latency and enables autonomous mid-flight threat response.
P3	Active Learning Feedback Loop	Allow operators to correct false positives and negatives via the dashboard. Store corrections as labelled examples and use them to periodically fine-tune alert rule thresholds and, over time, the VLM itself.

Table 8.1 — Prioritised future improvement roadmap.

9. Conclusion

The Drone Security Analyst Agent demonstrates that production-grade AI security monitoring is achievable using entirely open-source, locally-running components. The Hybrid Intelligence architecture — combining deterministic rule-based alerting with contextual VLM perception and agentic LLM reasoning — addresses the fundamental tension in security AI between the requirement for instant response on known threats and the need for deep contextual analysis of novel situations.

The prototype successfully fulfils all three key requirements: it processes simulated drone telemetry and video frames concurrently, generates real-time security alerts with five-level severity classification, and exposes a natural-language Q&A; interface grounded in a fully indexed session history. All capabilities function offline on a consumer laptop without external API dependencies.

The most significant technical contribution of this assignment was the decoupled pipeline architecture. By isolating perception, alerting, reasoning, and storage into independent layers, each component could be developed, tested, and iterated upon independently — and AI development tools could be applied most effectively at the scaffolding stage while preserving human control over all security-critical logic.

The system is designed with a clear production upgrade path: the VLM adapter, the WebSocket manager, and the session storage layer are all structured for replacement without cascading changes — ensuring that the prototype can evolve into a production-grade system as requirements mature.

Tanay Singh — B.Tech CSE (AI & ML), SPSU Udaipur — GitHub: tanaysingh0312
Report submitted: 14 May 2026 — Generated: 14 May 2026